Cours 08 — Les `struct` : regrouper ce qui va ensemble

> **Avant** : cours 06 bis (`if` , `for` , `rand`), cours 06 ter (`string` , `vector` , objets), cours 07 (la `struct Position` , les paramètres copiés). **Comment travailler** : deux **programmes console** (pas d'openFrameworks), un par partie. Dans Visual Studio, crée un projet *Application console* pour le combat Pokémon (tout dans `main.cpp`), puis un **deuxième projet** pour le dossier étudiant — c'est lui qu'on découpera en plusieurs fichiers.

Au cours 07, tu as aperçu la `struct` : deux `float` collés dans une boîte `Position` . Ce cours en fait l'outil central. Un Pokémon, c'est un nom, un type, un niveau, des points de vie, cinq statistiques et une liste d'attaques : quatorze informations qui **vont ensemble**. La `struct` les range dans une seule boîte à compartiments — et en deuxième partie, elle apprendra aussi à **faire** des choses, avec ses propres fonctions.

Première partie : un combat Pokémon, dans un seul fichier. Deuxième partie : un dossier étudiant, découpé en plusieurs fichiers, comme un vrai projet.

1. Le problème : des variables en vrac

Sans `struct` , deux Pokémon ressemblent à ça :

```
string dracaufeuName;
string dracaufeuType;
int dracaufeuLevel;
int dracaufeuHp;
// ... et encore cinq statistiques, puis tout pareil pour pikachu :
string pikachuName;
int pikachuHp;
// ...
```

Dix-huit variables, des noms à rallonge, et rien n'empêche de mélanger les PV de l'un avec le niveau de l'autre. Tu as déjà rencontré ce problème au cours 07 : les deux listes parallèles `xCoords` / `yCoords` , à toujours modifier ensemble — et la `struct Position` qui les fusionnait. Même remède ici, en plus grand.

2. Étape 1 — la fiche du Pokémon

Dans un nouveau projet console, avant `main()` :

```
#include <string>
#include <vector>
#include <iostream>

using std::string;
using std::vector;
using std::cout;
using std::endl;

struct Pokemon {
    string name;
    string type;
    int level;
    int hp;

    int attack;
    int attackSpe;
    int defense;
    int defenseSpe;
    int speed;
};
```

Et dans `main()`, on crée un Pokémon et on remplit sa fiche, compartiment par compartiment :

```
int main() {
    Pokemon dracaufeu{};

    dracaufeu.name = "Dracaufeu";
    dracaufeu.type = "Feu";
    dracaufeu.level = 50;
    dracaufeu.hp = 100;
    dracaufeu.attack = 84;
    dracaufeu.attackSpe = 109;
    dracaufeu.defense = 78;
    dracaufeu.defenseSpe = 109;
    dracaufeu.speed = 50;

    cout << dracaufeu.name << " (niveau " << dracaufeu.level << ", "
         << dracaufeu.hp << " PV)" << endl;

    return 0;
}
```

Relisons :

- `struct Pokemon { ... };` **fabrique un type**, comme `int` ou `string` — mais à toi. Par convention, son nom prend une majuscule. Le point-virgule après l'accolade fermante est obligatoire (oubli classique).
- `Pokemon dracaufeu{};` crée une **boîte à compartiments** : une case `string` pour le nom, une case `int` pour le niveau… toutes collées en mémoire, comme la `Position` du cours 07. Les accolades vides `{}` mettent les nombres à zéro — sans elles, les cases naissent avec un contenu inconnu (cours 01).
- `dracaufeu.name` ouvre un compartiment avec le **point** — la même syntaxe que `p.x` au cours 07, et que `noms.size()` depuis le cours 04.

> **Essaie** : ajoute un champ `string surnom;` à la fiche, remplis-le, affiche-le. Puis retire les `{}` de la création et affiche `dracaufeu.level` **sans** le remplir : qu'y a-t-il dans la case ?

3. Étape 2 — une struct dans la struct : les attaques

Un Pokémon a des attaques, et une attaque a elle-même plusieurs caractéristiques. Donc : une deuxième `struct`, et une **liste** de cette struct dans la fiche du Pokémon. Avant `struct Pokemon` :

```
struct Move {
    string name;
    string type;           // type élémentaire : Feu, Electrik...
    int power;
    int accuracy;
    int pp;                // points de puissance restants
    int ppMax;
    float criticalHitRate; // taux de coup critique, de 0.0 à 1.0
};
```

Et dans `struct Pokemon`, un compartiment de plus, tout en bas :

```
    vector<Move> moves;    // la liste des attaques du Pokémon
```

Dans `main()`, on fabrique les attaques et on les range — et on crée Pikachu au passage :

```
Move lanceFlammes{ "Lance-Flammes", "Feu", 90, 100, 15, 15, 0.0f };
Move dracogriffe{ "Dracogriffe", "Dragon", 70, 100, 25, 25, 0.1f };
dracaufeu.moves.push_back(lanceFlammes);
dracaufeu.moves.push_back(dracogriffe);

Pokemon pikachu{};
pikachu.name = "Pikachu";
pikachu.type = "Electrik";
pikachu.level = 25;
pikachu.hp = 35;
pikachu.attack = 55;
pikachu.attackSpe = 50;
pikachu.defense = 40;
pikachu.defenseSpe = 50;
pikachu.speed = 90;
Move eclair{ "Eclair", "Electrik", 40, 90, 25, 25, 0.2f };
Move viveAttaque{ "Vive-Attaque", "Normal", 30, 100, 40, 40, 0.1f };
pikachu.moves.push_back(eclair);
pikachu.moves.push_back(viveAttaque);
```

Deux nouveautés :

- `Move lanceFlammes{ "Lance-Flammes", "Feu", 90, 100, 15, 15, 0.0f };` remplit **tous les compartiments d'un coup**, dans l'ordre de la déclaration — la liste d'initialisation du cours 06 ter, version struct. Plus court que neuf affectations, mais l'**ordre des champs** compte : le premier nombre va dans `power` parce que `power` est déclaré en premier.
- `dracaufeu.moves[0].name` se lit de **gauche à droite** : dans `dracaufeu`, ouvre `moves`, prends l'élément 0, ouvre `name`. Des boîtes dans des boîtes — et le point à chaque étage.

> **Essaie** : affiche la première attaque de chaque Pokémon avec ses PP, au format `Lance-Flammes (15/15 PP)`.

4. Étape 3 — le scénario d'attaque

Le combat commence. Le plus **rapide** attaque en premier :

```
cout << "Debut du combat entre " << dracaufeu.name << " et " << pikachu.name << " !" << endl;
cout << "----------------------" << endl << endl;

Pokemon premierAttaquant{};
Pokemon deuxiemeAttaquant{};

if (dracaufeu.speed > pikachu.speed) {
    premierAttaquant = dracaufeu;
    deuxiemeAttaquant = pikachu;
}
else {
    premierAttaquant = pikachu;
    deuxiemeAttaquant = dracaufeu;
}
```

Arrête-toi sur `premierAttaquant = pikachu;` : l'affectation copie **toute la fiche** — le nom, les neuf nombres, et même la liste d'attaques. Une struct se manipule comme un `int` : on l'affecte, on la copie, d'un seul geste. (Conséquence : le combat abîmera les **copies** ; `dracaufeu` et `pikachu`, les originaux, resteront intacts.)

Puis le premier attaquant choisit une attaque **au hasard** dans sa liste, et on calcule les dégâts :

```
    cout << premierAttaquant.name << " va attaquer !" << endl;

    // choisir une attaque au hasard dans la liste
    int nbAttaques = premierAttaquant.moves.size();
    int indiceAttaque = rand() % nbAttaques;
    Move attaqueChoisie = premierAttaquant.moves[indiceAttaque];
    cout << premierAttaquant.name << " utilise " << attaqueChoisie.name << " !" << endl;

    // les dégâts : puissance de l'attaquant contre défense de la cible
    float puissanceAttaque = premierAttaquant.attack * attaqueChoisie.power;
    float defenseCible = 0;
    if (attaqueChoisie.type == "Normal") {
        defenseCible = deuxiemeAttaquant.defense;
    }
    else {
        defenseCible = deuxiemeAttaquant.defenseSpe;
    }
    float degats = puissanceAttaque / defenseCible;
    int degatsArrondis = (int)degats;

    cout << premierAttaquant.name << " inflige " << degatsArrondis
         << " points de degats a " << deuxiemeAttaquant.name << " !" << endl;
    deuxiemeAttaquant.hp = deuxiemeAttaquant.hp - degatsArrondis;
    cout << deuxiemeAttaquant.name << " a maintenant " << deuxiemeAttaquant.hp
         << " points de vie." << endl << endl;
```

Tout vient des cours précédents : `rand() % n` (cours 06 bis — pense à planter la graine `srand(static_cast<unsigned int>(time(0)));` en tête de `main`, avec `#include <cstdlib>` et `#include <ctime>`, sinon le combat est identique à chaque lancement), le `if` sur le type d'attaque, la division. Seule nouveauté : `(int)degats` **convertit** le `float` en `int` — la partie après la virgule est jetée, comme dans la division entière du cours 01.

> **Essaie** : fais riposter le deuxième attaquant — recopie le bloc d'attaque en échangeant les deux rôles. Oui, c'est du copier-coller douloureux : c'est voulu, les étapes 4 et 5 vont arranger ça.

5. Étape 4 — la boucle de combat

Un seul échange, ce n'est pas un combat. On enferme l'aller-retour dans une boucle `for` : dix tours maximum, et on s'arrête avant si un Pokémon tombe K.O. :

```
    for (int tour = 1; tour <= 10; tour++) {
        cout << "--- Tour " << tour << " ---" << endl;

        // [ici, le bloc : premierAttaquant attaque deuxiemeAttaquant]

        if (deuxiemeAttaquant.hp <= 0) {
            cout << deuxiemeAttaquant.name << " est K.O. ! "
                 << premierAttaquant.name << " gagne !" << endl;
            break;
        }

        // [ici, le même bloc, rôles échangés : deuxiemeAttaquant riposte]

        if (premierAttaquant.hp <= 0) {
            cout << premierAttaquant.name << " est K.O. ! "
                 << deuxiemeAttaquant.name << " gagne !" << endl;
            break;
        }
    }
```

Le `for` compte les tours, les `if` surveillent les PV, et le `break` (cours 06 bis) termine le combat dès qu'un camp tombe. Lance plusieurs fois : les attaques tirées au hasard changent le déroulement.

Mais regarde ton `main()` : le bloc d'attaque y est écrit **deux fois**, à l'identique, aux rôles près. Au cours 07, ce symptôme avait un remède : la **fonction**.

> **Essaie** : passe à 3 tours maximum, puis donne 300 PV aux deux Pokémon — le message de fin de boucle sans K.O. manque ! Ajoute, après le `for`, un affichage des PV restants des deux camps.

6. Étape 5 — des fonctions qui reçoivent des Pokémon

Dans un programme console, pas de `ofApp::` : on écrit des fonctions **globales**, avant `main()`. Et une struct se passe en paramètre exactement comme un `int` :

```
// Choisit une attaque au hasard dans la liste de l'attaquant
Move choisirAttaque(Pokemon attaquant) {
    int nbAttaques = attaquant.moves.size();
    int indiceAttaque = rand() % nbAttaques;
    return attaquant.moves[indiceAttaque];
}

// Calcule les dégâts d'une attaque de l'attaquant sur la cible
int calculerDegats(Pokemon attaquant, Move attaque, Pokemon cible) {
    float puissanceAttaque = attaquant.attack * attaque.power;
    float defenseCible = 0;
    if (attaque.type == "Normal") {
        defenseCible = cible.defense;
    }
    else {
        defenseCible = cible.defenseSpe;
    }
    return (int)(puissanceAttaque / defenseCible);
}
```

Le bloc d'attaque de la boucle fond à quatre lignes — à écrire deux fois, une par sens :

```
        Move attaque = choisirAttaque(premierAttaquant);
        int degats = calculerDegats(premierAttaquant, attaque, deuxiemeAttaquant);
        cout << premierAttaquant.name << " utilise " << attaque.name << " et inflige "
             << degats << " degats a " << deuxiemeAttaquant.name << " !" << endl;
        deuxiemeAttaquant.hp = deuxiemeAttaquant.hp - degats;
```

Deux choses à bien voir :

- Ces fonctions **renvoient** leur résultat avec `return` — une `Move`, un `int` — comme `rand()` au cours 06 bis. Le type devant le nom de la fonction (`Move`, `int`) annonce ce qu'elle renvoie ; `void` (cours 07) voulait dire « rien ».
- Pourquoi `calculerDegats` ne baisse-t-elle pas les PV de la cible elle-même ? Parce que ses paramètres sont des **copies** (cours 07) : à l'intérieur, `cible` est une photocopie de la fiche, et `cible.hp = ...` ne toucherait que la photocopie. La fonction **calcule et renvoie** ; c'est l'appelant qui applique le résultat sur la vraie variable.

Piège : c'est l'erreur la plus fréquente avec les structs — écrire `void attaquer(Pokemon cible) { cible.hp = cible.hp - 10; }`, l'appeler, et constater que rien ne change. La fonction a bien baissé les PV… de sa copie, morte à l'accolade fermante. Tant qu'on ne sait pas faire autrement, la règle

est : **la fonction renvoie, l'appelant modifie.** La deuxième partie du cours montre une vraie solution.

Essaie : écris `void afficherEtat(Pokemon p)` qui affiche `nom : X PV`, et appelle-la pour les deux camps à la fin de chaque tour. Une fonction d'affichage, elle, se moque de recevoir une copie — pourquoi ?

Exercices

1. **Lire avant de lancer** — que s'affiche-t-il, et pourquoi ?

```
Pokemon a{};
a.name = "Salameche";
a.hp = 39;

Pokemon b = a;
b.hp = 0;

cout << a.name << " : " << a.hp << " PV" << endl;
```

2. **Le coup critique** — la fiche `Move` a un champ `criticalHitRate` (de 0.0 à 1.0) que le combat ignore. Utilise-le : à chaque attaque, tire un nombre entre 0 et 99 (`rand() % 100`) ; s'il est plus petit que `criticalHitRate * 100`, les dégâts sont doublés et on affiche « Coup critique ! ». À ranger dans `calculerDegats`.
3. **Les PP** — chaque utilisation d'une attaque doit consommer 1 PP, et une attaque à 0 PP ne doit plus être choisie. Attention : `choisirAttaque` renvoie une **copie** de la `Move` — pour décompter le vrai PP, il te faut l'**indice** de l'attaque dans la liste. À toi de réorganiser.
4. **L'équipe** *(plus costaud)* — un `vector<Pokemon> equipe` avec trois Pokémon, et une boucle `for` qui affiche l'équipe au format `Pikachu (niveau 25) : 2 attaques`. C'est le ruban du cours 07 : des boîtes à compartiments alignées dans une liste — et chaque boîte contient elle-même une liste.

La deuxième partie — le dossier étudiant : ranger un vrai projet

Le combat tenait dans un seul fichier, déjà bien rempli. Un vrai programme grossit : si tout reste dans `main.cpp`, il devient un mur de cinq cents lignes où plus personne ne retrouve rien. Deuxième projet, donc — un **gestionnaire d'étudiants** : un étudiant a une identité, une adresse, une promotion, et des proches à contacter… qui ont chacun leur propre adresse.

Deux nouveautés, et c'est tout le programme de cette partie : **un fichier par struct**, et des **fonctions à l'intérieur des structs**.

7. Étape 6 — un fichier par struct

Crée un nouveau projet console. Dans l'explorateur de solutions, clic droit sur le projet → *Ajouter* → *Nouvel élément* → **fichier d'en-tête** `Address.h` :

```
#pragma once
#include <string>
using std::string;

struct Address {
    int number;
    string street;
    int postalCode;
    string city;
};
```

Et dans `main.cpp` :

```
#include <iostream>
#include "Address.h"

using std::cout;
using std::endl;

int main()
{
    Address addressMaman{ 10, "Rue de la Paix", 75002, "Paris" };
    cout << addressMaman.number << " " << addressMaman.street << ", "
         << addressMaman.postalCode << " " << addressMaman.city << endl;
    return 0;
}
```

Ce que font ces deux lignes nouvelles :

- `#include "Address.h"` — avec des **guillemets**, pas des chevrons : « va chercher ce fichier dans mon projet » (les chevrons `<iostream>` cherchent dans la bibliothèque standard). L'include **colle** le contenu du fichier à cet endroit : `main.cpp` voit la définition de `Address` comme si elle était écrite chez lui.
- `#pragma once` — « ne me colle qu'une seule fois ». Quand plusieurs fichiers incluront `Address.h` par des chemins différents, sans cette ligne la struct serait définie deux fois, et le compilateur

refuserait.

Tu connais ce découpage depuis le premier jour : ton projet openFrameworks fait exactement ça avec `ofApp.h` (les déclarations) et `ofApp.cpp` (les définitions). Ici, on le fait **par struct** : chaque type a son fichier, et un projet se lit comme un sommaire.

> **Essaie** : ajoute une deuxième adresse (`20, "Avenue des Champs-Elysees", 75008, "Paris"`) et affiche les deux.

8. Étape 7 — des structs dans des structs, à travers les fichiers

Le dossier se construit par poupées russes. `Relative.h` — un proche, qui **contient** une adresse :

```
#pragma once
#include "Address.h"

struct Relative {
    string name;
    string firstName;
    string relationship;
    Address address;
};
```

`Promotion.h` :

```
#pragma once
#include <string>
using std::string;

struct Promotion {
    string name;
    string specialization;
    int year;
};
```

Et `Student.h` , qui assemble tout :

```
#pragma once
#include <vector>
#include "Relative.h"
#include "Promotion.h"

using std::vector;

struct Student {
    string name;
    string firstName;
    int age;
    int studentNumber;
    Address address;
    Promotion promotion;
    vector<Relative> relatives;
};
```

Regarde la **chaîne d'includes** : `Student.h` inclut `Relative.h`, qui inclut `Address.h`. Chaque fichier n'inclut que ce dont **il** a besoin, et le `#pragma once` garantit qu'`Address` ne sera collée qu'une fois, même si elle arrive par deux chemins. Dans `main.cpp`, un seul `#include "Student.h"` suffit alors pour tout avoir :

```
    Relative maman{};
    maman.address = addressMaman;
    maman.firstName = "Hortensia";
    maman.name = "Blaise";
    maman.relationship = "Maman";

    Promotion promotion{};
    promotion.name = "L3 Informatique";
    promotion.specialization = "Informatique";
    promotion.year = 2024;

    Student gaetan{};
    gaetan.promotion = promotion;
    gaetan.firstName = "Gaetan";
    gaetan.name = "Blaise";
    gaetan.age = 20;
    gaetan.studentNumber = 123456;
    gaetan.relatives.push_back(maman);

    cout << "Ma mere s'appelle : " << gaetan.relatives[0].firstName << endl;
```

Et l'accès en chaîne traverse les étages sans effort : `gaetan.relatives[0].address.city` — dans `gaetan`, le proche numéro 0, son adresse, sa ville.

> **Essaie** : crée le papa (`Jean Cazalet` , à la deuxième adresse), ajoute-le aux proches, et affiche la ville de chaque proche avec une boucle `for` sur `gaetan.relatives` .

9. Étape 8 — une fonction DANS la struct

Ajouter un proche, c'est toujours le même rituel : le `push_back`, et parfois recopier son adresse chez l'étudiant (quand il vit à la même adresse). En première partie, on aurait écrit une fonction globale — qui aurait reçu une **copie** de l'étudiant et n'aurait rien pu modifier. La vraie solution : mettre la fonction **dans** la struct. Dans `Student.h`, à l'intérieur de l'accolade :

```
struct Student {
    // ... les champs, comme avant ...
    vector<Relative> relatives;

    void addRelative(Relative relative, bool studentLivesAtSameAddress);
};
```

C'est l'**annonce**, comme au cours 07. La définition, elle, va dans un nouveau fichier `Student.cpp` :

```
#include "Student.h"

void Student::addRelative(Relative relative, bool studentLivesAtSameAddress) {
    relatives.push_back(relative);

    if (studentLivesAtSameAddress) {
        address = relative.address;
    }
}
```

Et dans `main()` :

```
    gaetan.addRelative(maman, true);
    gaetan.addRelative(papa, false);
```

Tout est déjà familier, assemblé autrement :

- `Student::addRelative` — le nom de la struct, deux fois deux-points, le nom de la fonction : tu écris `ofApp::draw` depuis le cours 05. Ça dit « cette fonction appartient à `Student` ».

- Dans son corps, `relatives` et `address` **sans point ni préfixe** : une fonction membre travaille **dans la boîte**. Quand on appelle `gaetan.addRelative(...)`, c'est la liste de `gaetan` qui s'allonge, l'adresse de `gaetan` qui change — pas une copie. C'est la réponse au Piège de la première partie.
- `gaetan.addRelative(maman, true)` — l'objet, le point, la fonction : la syntaxe de `noms.push_back(...)` depuis le cours 04. La boucle est bouclée : au cours 06 ter, on t'a dit qu'un **objet** combine des données et des fonctions membres. Tu viens d'en fabriquer un.

> **Essaie** : dans `addRelative`, ajoute `relative.firstName = "Gilbert";` **après** le `push_back`, relance, et affiche le prénom de la maman : il n'a pas bougé — ni chez `maman`, ni dans la liste. Compte les copies en jeu, puis retire la ligne.

10. Étape 9 — `setIdentity` : trois champs d'un coup

Les fonctions membres servent aussi à raccourcir les rituels de remplissage. Dans `Student.h` :

```
    void setIdentity(string nameP, string firstNameP, int ageP);
```

Dans `Student.cpp` :

```
void Student::setIdentity(string nameP, string firstNameP, int ageP) {
    name = nameP;
    firstName = firstNameP;
    age = ageP;
}
```

Et les trois affectations du `main()` deviennent une ligne :

```
    gaetan.setIdentity("Blaise", "Gaetan", 20);
```

Détail de nommage : les paramètres s'appellent `nameP`, `firstNameP` — avec un suffixe — pour ne pas porter **le même nom** que les champs qu'ils remplissent. Si le paramètre s'appelait `name`, la ligne `name = name;` serait ambiguë (et ne ferait rien d'utile).

> **Essaie** : écris `void afficher();` dans `Student` — sans paramètre : elle a déjà tout sous la main — qui affiche `Prenom Nom (numero), promotion X`. Appelle-la : `gaetan.afficher();`.

Exercices

1. **Lire avant de lancer** — que s'affiche-t-il, et pourquoi ?

```
Address a{ 10, "Rue de la Paix", 75002, "Paris" };
Relative papa{};
papa.address = a;
a.city = "Lyon";

cout << papa.address.city << endl;
```

2. **Le téléphone** — un `Phone.h` : une struct `PhoneCamera` (`int resolution`, `bool hasFlash`), puis une struct `Phone` (`string model`, `int variant`, `vector<string> accessories`, et une `PhoneCamera camera`). Dans `main()`, crée un téléphone, remplis tout, ajoute trois accessoires, et affiche sa fiche — la liste d'accessoires avec une boucle.
3. **La fiche complète** — enrichis `afficher()` (l'Essaie de l'étape 9) pour qu'elle imprime **tout** le dossier : identité, adresse, promotion, et chaque proche avec son lien de parenté — une boucle sur `relatives`, dans la fonction membre.
4. **La liste d'appel** *(plus costaud)* — dans `main()`, un `vector<Student>` avec trois étudiants. Ajoute à `Student` une fonction membre `bool estMajeur();` (elle **renvoie** un `bool`, cours 06 bis), puis une boucle qui affiche chaque étudiant suivi de `majeur` ou `mineur`. Trois structs, deux fichiers, une boucle, un `return` : toute la leçon en douze lignes.

Ce qu'il faut retenir

- `struct Nom { ... };` fabrique un **type à compartiments** ; majuscule par convention, point-virgule final obligatoire. On ouvre un compartiment avec le **point**.
- `Pokemon p{};` initialise les cases à zéro ; `Move m{ "Eclair", "Electrik", 40, ... };` remplit tout d'un coup, **dans l'ordre des champs**.
- Une struct se manipule comme un `int` : l'affectation et le passage en paramètre **copient toute la fiche**. Une fonction globale ne peut donc pas modifier l'original : elle **renvoie** son résultat (`return`), et l'appelant applique.
- Les structs s'emboîtent — une struct dans une struct, un `vector` de structs — et l'accès se lit de gauche à droite : `gaetan.relatives[0].address.city`.
- Un vrai projet se range : **un fichier `.h` par struct** (`#pragma once`, includes en chaîne, guillemets pour tes fichiers), les définitions dans le `.cpp` — le découpage de `ofApp.h` / `ofApp.cpp` depuis le premier jour.
- Une **fonction membre** se déclare dans la struct, se définit `void Student::f(...)` dans le `.cpp`, et travaille **dans la boîte** : elle lit et modifie les champs de l'objet sur lequel on l'appelle — `gaetan.addRelative(...)`, comme `noms.push_back(...)` depuis toujours.
- Données + fonctions membres = un **objet** (cours 06 ter). Tu sais maintenant en fabriquer.